EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Mini Project 2

Student Name	Hritvik Kuppa
NUID	003947137
Email	kuppa.h@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	11/03/2026

Submission Checklist (must match syllabus requirements)

- Submit **one PDF** generated from this Overleaf report.
- Include **clear screenshots** for each task (text must be readable).
- Include your **GitHub profile link** and **Overleaf project link** in the report.

How to complete this assignment

1. Complete each task below in order.
2. For each task, include the required evidence (screenshots, links, and short notes).
3. When you paste a link, ensure it opens correctly (no private/local-only links unless instructed).

Task 1

INSERT

```
1:  function INSERT(a)
2:      parent ← NULL
3:      curr ← root
4:      while curr ≠ NULL do
5:          parent ← curr
6:          if a.iata < curr.iata then
7:              curr ← curr.left
8:          else if a.iata > curr.iata then
9:              curr ← curr.right
10:         else
11:             return FALSE                                  ▷ I4 duplicate
12:         end if
13:     end while
14:     create newNode with a
15:     if parent = NULL then
16:         root ← newNode                                    ▷ I1
17:     else if a.iata < parent.iata then
18:         parent.left ← newNode                             ▷ I2
19:     else
20:         parent.right ← newNode                            ▷ I3
21:     end if
22:     count ← count + 1
23:     return TRUE
24: end function
```

BUILD-FROM-ARRAY

```
1: function BUILD-FROM-ARRAY(data, n)
2:     dup ← 0
3:     for i ← 0 to n − 1 do
4:         if INSERT(data[i]) = FALSE then
5:             dup ← dup + 1
6:         end if
7:     end for
8: end function
```

HEIGHT AND DEPTH

```
1: function HEIGHT-AND-DEPTH( )
2:     if root = NULL then
3:         return −1, 0
4:     end if
5:     PUSH root into queue Q
6:     depth ← 0
7:     total ← 0
8:     while Q is not empty do
9:         n ← Q.size
10:        for i ← 0 to n − 1 do
11:            curr ← Q.pop
12:            total ← total + depth
13:            PUSH the non-NULL children of curr into Q
14:        end for
15:        depth ← depth + 1
16:    end while
17:    return depth − 1, total/count
18: end function
```

Task 2

SEARCH

```
1: function SEARCH(code)
2:     curr ← root
3:     comp ← 0
4:     while curr ≠ NULL do
5:         APPEND curr.iata to path
6:         comp ← comp + 1
7:         if code < curr.iata then
8:             curr ← curr.left
9:         else if code > curr.iata then
10:            curr ← curr.right
11:        else
12:            return curr                                  ▷ found
13:        end if
14:    end while
15:    return NULL                                          ▷ not found
16: end function
```

MIN / MAX

```
1: function MIN( )
2:     if root = NULL then
3:         return NULL
4:     end if
5:     curr ← root
6:     while curr.left ≠ NULL do
7:         curr ← curr.left
8:     end while
9:     return curr
10: end function

11: function MAX( )
12:     if root = NULL then
13:         return NULL
14:     end if
15:     curr ← root
16:     while curr.right ≠ NULL do
17:         curr ← curr.right
18:     end while
19:     return curr
20: end function
```

Task 3

DELETE

```
1:  function DELETE(code)
2:      parent ← NULL
3:      curr ← root
4:      while curr ≠ NULL and curr.iata ≠ code do
5:          parent ← curr
6:          if code < curr.iata then
7:              curr ← curr.left
8:          else
9:              curr ← curr.right
10:         end if
11:     end while
12:     if curr = NULL then
13:         return FALSE                                   ▷ not found
14:     end if
15:     if curr.left ≠ NULL and curr.right ≠ NULL then     ▷ D3
16:         s ← curr
17:         succ ← curr.right
18:         while succ.left ≠ NULL do                      ▷ inorder successor
19:             s ← succ
20:             succ ← succ.left
21:         end while
22:         curr.data ← succ.data                          ▷ copy it
23:         parent ← s
24:         curr ← succ                                    ▷ delete the successor
25:     end if
26:     if curr.left ≠ NULL then                           ▷ D1 / D2
27:         child ← curr.left
28:     else
29:         child ← curr.right
30:     end if
31:     if parent = NULL then
32:         root ← child
33:     else if parent.left = curr then
34:         parent.left ← child
35:     else
36:         parent.right ← child
37:     end if
38:     delete curr
39:     count ← count − 1
40:     return TRUE
41: end function
```

LeftRightChild

```
1: function CHILDREN(left, right)
2:     PUSH root into queue Q
3:     while Q is not empty do
4:         curr ← Q.pop
5:         if curr ≠ root and (curr.left ≠ NULL) = left and (curr.right ≠ NULL) = right
   then
6:             return curr.iata
7:         end if
8:         PUSH the non-NULL children of curr into Q
9:     end while
10:    return “”
11: end function
```

PRINT LEVELS

```
1: function PRINT-LEVELS( )
2:     PUSH root into queue Q
3:     depth ← 0
4:     while Q is not empty and depth < 3 do
5:         n ← Q.size
6:         for i ← 0 to n − 1 do
7:             curr ← Q.pop
8:             PRINT(curr.iata)
9:             PUSH the non-NULL children of curr into Q
10:        end for
11:        depth ← depth + 1
12:    end while
13: end function
```

Task 4

INORDER

```
1: function INORDER( )
2:     curr ← root
3:     empty stack S
4:     while curr ≠ NULL or S is not empty do
5:         while curr ≠ NULL do
6:             PUSH curr onto S
7:             curr ← curr.left
8:         end while
9:         curr ← S.pop
10:        APPEND curr to list
11:        curr ← curr.right
12:    end while
13: end function
```

CHECK-SORTED

```
1: function CHECK-SORTED(list, n)
2:     for i ← 0 to n − 2 do
3:         if list[i].iata > list[i + 1].iata then
4:             return FALSE
5:         end if
6:     end for
7:     return TRUE
8: end function
```